Decision Agent 技术定位与演进规划

> 文档版本：v0.2 · 2026-04-28
> 适用范围：kweaver-core + kweaver-dip

一、背景

经过 0.7.x 版本的迭代，Decision Agent 已具备 BKN 语义建模、Context Loader 精准召回、Execution Factory 沙盒执行以及 TraceAI 审计留痕的基础能力。

Decision Agent 与 OpenClaw 的关系：两者的核心差异不在于"谁能长期运行"，而在于**治理深度与使用场景**。Decision Agent 是治理优先的编排底座——所有决策与执行都经过 BKN 语义护栏、ISF 权限控制与 TraceAI 审计，适合企业级多 Agent 协同任务；OpenClaw 是轻量级个人助理，强调快速响应和个人效率。两者都应具备 7×24 能力，但 Decision Agent 带完整治理链路，OpenClaw 更轻便直接。Decision Agent 也可独立使用，不依赖 OpenClaw 即可完成单体复杂任务。

在向"企业生产级多智能体系统"演进的过程中，Decision Agent 暴露出四个结构性短板：

瓶颈	现象	根因
非 7×24 设计	Decision Agent 能执行长时间任务，但缺乏轻量常驻 Runtime、定时任务、心跳保活等机制，无法像真实员工一样持续在线	架构初期面向请求-响应模式设计，缺少进程内 Scheduler、任务状态持久化、任务认领与防重等 7×24 基础能力
使用门槛高	多 Agent 组合是 Decision Agent 的核心优势（相对于 OpenClaw），但用户需要手工选择、配置和串联多个 Agent，场景搭建成本高	缺乏基于任务意图自动选择、实例化和组合 Decision Agent 的上层能力，高级协同能力被配置门槛掩盖
协同信息孤岛	多个 Decision Agent 之间只能通过上下文拼接传递中间产物，Token 膨胀且语义失真	缺少面向 Agent Team 的共享工作区（Shared Workspace）机制
记忆体系不完整	已具备基础 Memory 写入与召回能力，但缺少分层记忆、去重合并、治理策略和与 Context Loader 的体系化融合	现有 Memory 更偏单 Agent/单用户的基础能力，尚未形成 User/Role/Org 分层沉淀与可运维的记忆生命周期

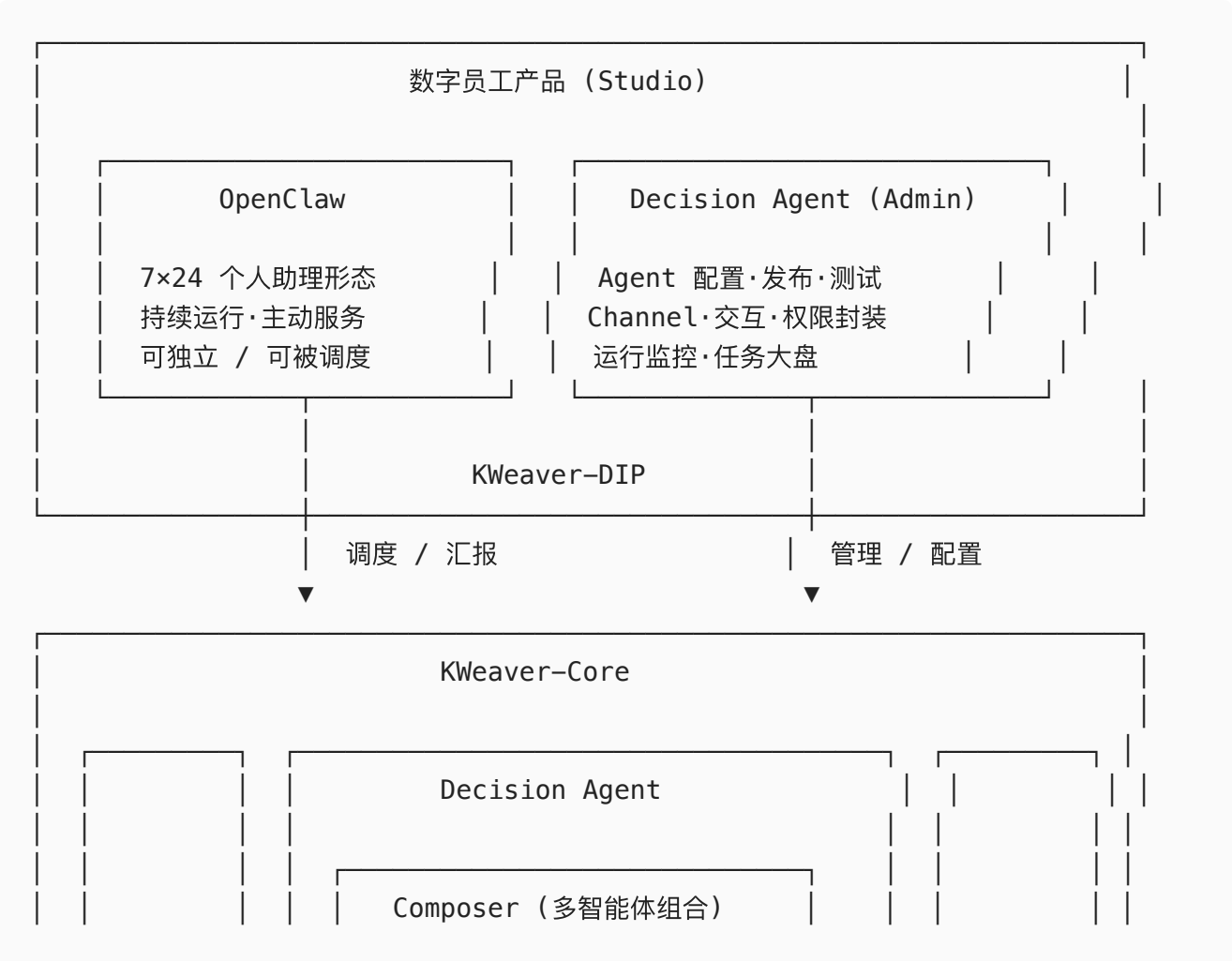
二、定位分析

2.1 外部竞争格局

竞品/框架	核心优势	Decision Agent 的差异化
Palantir AIP	数据本体 + Ontology SDK + 执行治理，已验证企业付费意愿	KWeaver 在同等治理深度下交付半径更大、部署门槛更低（信创适配、轻量部署）；BKN 的 Markdown 扩展语法让业务专家零代码参与建模，Palantir 依赖开发者
Dify / Coze	低代码 Agent Builder，快速搭建	止步于"RAG + Prompt + 工具调用"的浅层编排，缺少结构化上下文治理（Context Loader 级别的召回精排）与语义级安全管控（ISF），无法进入强监管场景
LangGraph	图状态机，精确控制分支与循环	纯开发者框架，无业务语义层；Decision Agent 将 BKN 语义网络作为 Agent 的"业务约束护栏"，而非仅靠 Prompt 约束行为
CrewAI	角色化 Agent 团队，快速原型	缺乏企业级治理（审计、权限、风险建模）；团队协作停留在"对话传递"层面，无结构化共享存储

Decision Agent 的核心差异化主张：不是又一个 Agent 框架，而是**治理优先、Harness-first 的企业级决策智能体底座**——BKN 提供业务语义护栏，ISF 提供安全边界，TraceAI 提供审计底座，Context Loader 提供高质量上下文。这些能力在 Dify/CrewAI 等框架中不存在，在 Palantir 中以更高成本存在。

2.2 内部架构边界



```
│  │         │  │  └──────────────────────────┘    │  │         │ │
│  │         │  │                ▼                  │  │         │ │
│  │  I S F  │  │  ┌──────────────────────────┐    │  │ Trace   │ │
│  │         │  │  │   Agent Factory (智能体工厂) │    │  │  A I    │ │
│  │  安全   │  │  └──────────────────────────┘    │  │         │ │
│  │  治理   │  │                ▼                  │  │  审计   │ │
│  │         │  │  ┌──────────────────────────┐    │  │  可观测 │ │
│  │         │  │  │ Dolphin Runtime (智能体引擎) │    │  │         │ │
│  │         │  │  └──────────────────────────┘    │  │         │ │
│  │         │  └──────────────────────────────────┘  │         │ │
│  │         │                                        └─────────┘ │
│  └─────────┘                                                    │
│                                                                 │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ ┌──────────────┐ │
│  │Context Loader│ │  BKN Engine  │ │  VEGA    │ │  Execution   │ │
│  │ 上下文治理   │ │ 业务知识网络 │ │  Engine  │ │  Factory     │ │
│  │概念·对象·精排│ │对象·关系·风险│ │数据虚拟化│ │ 沙盒化执行   │ │
│  └──────────────┘ └──────────────┘ └──────────┘ └──────────────┘ │
└─────────────────────────────────────────────────────────────────┘
```

关键边界说明：

- **Composer 是基于 Decision Agent 的上层组合能力**，不是独立通用 Agent 框架。它面向复杂任务动态生成 Agent Team Plan，并调度多个 Decision Agent 协同完成目标。
- **Composer 与 Dolphin 的关系**：Decision Agent 以 Dolphin（DPH）为执行内核；Dolphin 负责单个 Agent 内部的推理、工具调用与流程控制，Composer 负责多个 Decision Agent 的选择、实例化、分工与协作。Composer 不替代 Dolphin，也不以生成 Dolphin 脚本为第一目标。

三、待增强关键需求

3.1 开箱即用：降低单 Agent 使用门槛

要解决的问题：Decision Agent 已具备自然语言模式与 Dolphin 模式，但对初学者而言，Dolphin 脚本、工具绑定、上下文配置和调试流程仍然有较高门槛。业务团队在搭建单个可用 Agent 时，经常需要从头理解配置结构与 DPH 流程定义，导致第一次跑通业务场景的周期偏长。

目标：提供开箱即用的预置模板和运行模式，让用户在不编写复杂 DPH 脚本的前提下快速创建并运行单个 Decision Agent，降低从"第一次使用"到"跑通业务场景"的时间。多 Agent 的动态选择、实例化和组合由 Composer 解决。

核心需求：

- **预置运行模式**：提供 ReAct 模式（封装 /explore/，单 Agent 自主推理执行）、Plan-Execute 模式（先规划后执行）等开箱即用模式，用户通过配置选择而非编写脚本

- **场景模板库**：针对常见业务场景（如经营复盘、数据巡检、报告生成等）提供可直接使用或小幅修改的 Agent Template
- **渐进式学习路径**：从"选模板 - 改参数 - 写简单脚本 - 自定义 DPH"逐级过渡，每一步都有可运行的最小示例
- **可视化辅助**：在 DIP 管理端提供 Agent 配置、工具绑定、模板参数和调试结果的可视化辅助，降低单 Agent 搭建门槛

3.2 Heartbeat + Cron：轻量常驻 Runtime 与主动执行

要解决的问题：Agent 只能被动等待用户请求，无法像真实员工一样主动巡检、定时执行或在后台持续推进任务。

目标：Decision Agent 具备"永远在线"的能力——通过轻量常驻 Runtime 定期自我唤醒，主动检查并执行到期任务，结果异步投递给用户。Cron/Heartbeat 是 Decision Agent 常驻数字员工路线的核心运行能力；DIP Studio 负责计划任务的呈现、配置和管理，Decision Agent Runtime 负责计划语义、调度判断、任务认领、执行隔离、重试恢复、审计留痕与结果投递。

架构边界：

- **Runtime 内建轻量 Scheduler**：单实例形态下采用进程内 timer/event loop 即可，不强依赖 K8s CronJob 或云厂商调度器
- **DIP Studio 管理控制面**：负责计划创建、编辑、启停、运行历史、执行日志和告警展示，不承载任务执行
- **Decision Agent Runtime 执行面**：负责到期判断、执行模式选择、权限治理、沙盒执行、TraceAI 审计和结果投递

核心需求：

- **Heartbeat（心跳运行器）**：Runtime 按固定间隔自我唤醒 Agent，读取任务清单，有到期任务则执行，无到期任务则静默。支持活跃时段控制和夜间降频，避免无人值守时浪费资源
- **Cron（定时调度引擎）**：支持 Cron 表达式与简易 Interval 两种调度方式，任务声明采用人机双向可读格式，支持时区感知
- **任务状态持久化**：记录任务定义、下次运行时间、运行状态、重试次数、最近错误、运行历史与结果投递状态，支持 Runtime 重启后的恢复与补偿
- **执行隔离**：轻量任务与心跳共享上下文（inline），重量级任务在独立沙盒会话中执行（isolated），互不干扰
- **Reflection 能力**：心跳空闲时，Agent 审视记忆与近期上下文，主动发现重复性意图并提议新的定时任务
- **结果投递**：心跳结果通过 Mailbox 机制异步投递到用户主会话，不污染对话历史
- **故障容错**：支持重试、永久错误自动跳过、超时降级等策略

3.3 Shared Workspace：多智能体协同工作区

要解决的问题：多个 Decision Agent 协作时，中间分析结果、数据表、报告草稿、文件产物等只能通过上下文拼接传递，多步传参导致 Token 膨胀、语义失真和产物不可追踪。

目标：为同一 Session / Task 下的 Agent Team 提供共享工作区。各 Decision Agent 通过内置 Tool 读写中间产物，提示词中只传递引用与摘要，不传递全文，从根本上降低上下文压力，并让协作产物可追踪、可审计、可清理。

边界说明：

- **不同于 Context Loader**：Context Loader 负责从 BKN、数据源、文档、Memory 中召回业务上下文；Shared Workspace 负责当前任务内的中间产物交换
- **不同于 Memory**：Memory 负责跨会话沉淀偏好、纠错和经验；Shared Workspace 默认随 Session / Task 生命周期清理，不作为长期记忆
- **不同于 Dolphin 变量**：Dolphin 变量服务于单个 Decision Agent 内部流程；Shared Workspace 服务于多个 Decision Agent 之间的协作
- **不同于文件归档**：文件归档面向最终产物留存；Shared Workspace 面向执行过程中的中间产物与引用传递

核心需求：

- 以 Session / Task 为隔离边界，创建共享工作区
- 支持文本、JSON、表格、文件引用、报告草稿、执行结果等产物类型
- 提供 `workspace.put` / `workspace.get` / `workspace.list` / `workspace.delete` / `workspace.summarize` 等内置 Tool
- 写入时生成稳定引用 ID，Agent 间传递引用和摘要，而非全文
- 结合 ISF 实施读写权限控制，子 Agent 只能访问被授权的 namespace / key
- 支持 TTL、大小限制、类型限制和自动清理，防止工作区无限膨胀
- 支持 Trace AI 记录关键读写事件，便于审计协作链路

3.4 Composer：基于 Decision Agent 的轻量多智能体组合

要解决的问题：当前多 Agent 协同主要依赖人工选择、配置和串联多个 Decision Agent。面对复杂业务任务时，用户需要先判断需要哪些 Agent、如何分工、如何传递上下文、如何汇总结果，场景搭建成本高，难以体现 Decision Agent 作为企业级多智能体底座的组合优势。

定位：Composer 构建在 Decision Agent 之上。Decision Agent 以 Dolphin 为执行内核，负责单个智能体内部的推理、工具调用与流程控制；Composer 面向更高层的任务组织，负责根据复杂任务动态选择、实例化和调度多个 Decision Agent 协同完成目标。

目标：用户用自然语言描述任务，Composer 基于 agent-factory 中的已有 Agent、Agent Template、配置模式与 Agent Skill，生成可审查的 Agent Team Plan。Team 成员可以是已有 Decision Agent，也可以是从特定模板或模式动态实例化出的临时 Decision Agent。经用户确认后，Composer 调度这些 Decision Agent 执行并汇总结果。

架构原则：

- **组合与执行解耦**：Composer 负责生成和管理 Agent Team Plan，具体子任务仍由被调度的 Decision Agent 执行；每个 Decision Agent 内部继续由 Dolphin 内核承载
- **已有能力优先**：优先复用 agent-factory 中已有 Agent、Agent Template、Agent Skill 与配置模式；仅在任务需要时动态实例化临时 Decision Agent
- **轻量起步**：第一阶段聚焦任务拆解、Agent 选择/实例化、分工计划、基础协作与结果汇总，不建设通用 DAG 工作流引擎

核心需求：

- **任务拆解**：将复杂任务拆解为少量可委派子任务，明确每个子任务的目标、输入、输出和完成标准
- **Agent 选择与实例化**：从 agent-factory 管理的已有 Agent、Agent Template、配置模式和 Agent Skill 中选择或实例化候选 Decision Agent
- **团队计划**：生成 Lead / Worker 分工，明确每个 Worker 的角色、依赖关系、上下文需求与结果汇报格式
- **轻量协作模式**：第一阶段支持顺序执行、并行扇出、结果汇总三种基本模式
- **执行委派**：通过 agent-executor 调用被选中的或临时实例化的 Decision Agent；各 Agent 内部继续由 Dolphin 内核执行
- **上下文与治理**：结合 Context Loader、BKN Engine、ISF 与 Trace AI，对 Agent 实例化、任务委派、上下文装配、风险动作和执行链路进行约束与审计
- **人工确认**：Agent Team Plan 执行前必须可审查、可修改、可确认

第一阶段非目标：

- 不做通用 DAG 工作流引擎
- 不做任意层级动态 spawn 与递归委派
- 不做独立事件溯源恢复系统
- 不定义通用 A2A 协议
- 不把 Composer 定位为 Dolphin 脚本生成器

3.5 Memory：跨会话记忆体系增强

现状基础：Decision Agent 已具备基础 Memory 能力——在 Agent 配置启用后，可通过内置 `build_memory` / `search_memory` 工具完成对话记忆的写入与召回，并在运行结束后异步构建记忆。这说明 Memory 不是空白能力，而是已有基础链路。

要解决的问题：现有 Memory 能力仍偏基础，缺少企业级记忆体系所需的分层、治理和可运维机制。Agent 可以检索历史记忆，但尚不能稳定沉淀用户偏好、历史纠错、岗位经验和组织级经验，也缺少记忆去重、冲突合并、显式删除、衰减和审计等生命周期治理能力。

目标：在现有 Memory 链路上构建跨会话、可治理、可继承的持久化记忆体系，使 Agent 能沉淀经验、复用知识，新部署的数字员工可继承岗位历史经验。记忆写入采用自动提取关键事实、去重合并、持久化归档的机制。

核心需求：

- **保留并强化现有基础链路**：继续复用 `build_memory` / `search_memory` 工具和运行结束后的异步记忆构建机制，避免重建已有能力
- **短期记忆**：当前 Session 的交互历史、执行状态与中间决策，生命周期与会话一致
- **长期记忆**：跨 Session 持久化用户偏好、纠错记录、常用口径等，支持向量检索与结构化过滤
- **层级记忆**：User 级（个人偏好）→ Role 级（岗位经验）→ Org 级（组织知识），新员工 Agent 可继承岗位记忆
- **写入机制**：双通道写入（Agent 显式写入 + TraceAI/执行轨迹自动提取），并支持去重、合并与冲突处理
- **Memory 是 Context Loader 的数据源之一**：召回时从 Memory 中检索历史偏好，与 BKN 语义信息合并后精排输出
- **遗忘与治理**：支持基于频率的衰减、用户显式删除、敏感记忆屏蔽和审计追踪，防止记忆库无限膨胀

四、非目标（本轮迭代明确不做）

- **不做自研/蒸馏模型**：外部模型能力足够，飞轮数据量级达到支撑领域微调的规模时重新评估。
- **不做 Chat/Board UI 重大迭代**：UI 定位降级为附属，预算集中在 Agent-first 的核心能力上。DIP 侧只做最小可用的监控与配置界面。
- **不做通用 Agent 框架**：不与 LangGraph/CrewAI 竞争"通用编排框架"市场。Decision Agent 的价值在于 BKN 语义治理 + ISF 安全 + TraceAI 审计的垂直整合，而非通用性。
- **不做完全自动编排**：Composer 定位"半自动"——生成执行计划后必须支持人工审查与修改，不追求端到端全自动。
- **不自建重量级 Cron 基础设施**：Decision Agent 内建轻量 Scheduler Runtime，但不自研独立的分布式调度平台。单实例使用进程内 timer/event loop，多副本通过数据库/Redis/K8s Lease 等通用机制完成任务认领与防重。